Hodge diamond cutter

Release v1.2

Pieter Belmans

Aug 11, 2026

CONTENTS

1 Hodge diamonds **3**

2 Hochschild homology **17**

3 Constructions **21**

3.1 Elementary constructions 21
3.2 Curves and moduli spaces of sheaves on them 24
3.3 Surfaces and moduli spaces of sheaves on them 30
3.4 Abelian varieties and related objects 35
3.5 Fano varieties 35
3.6 Homogeneous varieties and closely related constructions 38
3.7 Moduli spaces attached to quivers 41
3.8 Hyperkähler varieties 42
3.9 Other 43

Python Module Index **45**

Index **47**

A tool to work with Hodge diamonds, comes with many varieties and constructions built into it.

- repository
- documentation

Hodge diamonds encode the Hodge numbers of a variety, and provide interesting information about its structure. They provide a numerical incarnation of many operations one can perform in algebraic geometry, such as blowups, projective bundles, products. They are also computed for many more specific constructions such as certain moduli spaces of sheaves, or flag varieties, ...

These Hodge numbers are defined as the dimensions of the sheaf cohomology of exterior powers of the cotangent bundle, i.e.

$$\mathrm{h}^{p,q}(X) = \dim \mathrm{H}^q(X, \Omega_X^p)$$

Here p and q range from 0 to $n=\dim X$. These numbers satisfy additional symmetry properties:

- Hodge symmetry: $\mathrm{h}^{p,q}(X)=\mathrm{h}^{q,p}(X)$
- Serre duality: $\mathrm{h}^{p,q}(X)=\mathrm{h}^{n-p,n-q}(X)$

Because of these symmetries they are usually displayed as a diamond (it's really just a square tilted 45 degrees), so that for a surface it would be:

```
                h^{2,2}
        h^{2,1}         h^{1,2}
h^{2,0}         h^{1,1}         h^{0,2}
        h^{1,0}         h^{0,1}
                h^{0,0}
```

One of their famous applications is the mirror symmetry prediction that every Calabi-Yau 3-fold has a mirror Calabi-Yau threefold, which should imply that their Hodge diamonds are transpositions. The first instance of this is the quintic 3-fold and its mirror, whose Hodge diamonds are:

```
              1
         0         0
    0         1         0
1       101       101       1
    0         1         0
         0         0
              1
```

and:

```
              1
         0         0
    0       101       0
1       1         1       1
    0       101       0
         0         0
              1
```

The following are some very basic examples of operations and constructions one can use within the Hodge diamond cutter. To get started we do:

```
sage: from diamond import *
```

after starting Sage.

Pretty print the Hodge diamond of a genus 2 curve:

```
sage: X = HodgeDiamond.from_matrix([[1, 2], [2, 1]])
sage: print(X)
      1
  2       2
      1
```

Compute the Euler characteristic of the product of X with itself:

```
sage: print((X*X).euler())
4
```

Pretty print the Hodge diamond of the Hilbert square of a K3 surface:

```
sage: S = HodgeDiamond.from_matrix([[1, 0, 1], [0, 20, 0], [1, 0, 1]])
sage: print(hilbn(S, 2))
                  1
              0       0
          1       21      1
      0       0       0       0
  1       21      232     21      1
      0       0       0       0
          1       21      1
              0       0
                  1
```

It also possible to generate LaTeX code:

```
sage: latex(K3().pprint())
\begin{tabular}{ccccc}
 &  & $1$ &  &  \\
 & $0$ &  & $0$ &  \\
$1$ &  & $20$ &  & $1$ \\
 & $0$ &  & $0$ &  \\
 &  & $1$ &  &  \\
\end{tabular}
```

There are many varieties built in, e.g. the previously defined K3 surface can be compared to the built-in one:

```
sage: print(S == K3())
True
```

Check out the documentation for all the available functionality.

If you use the software in your work, please cite it as explained on Zenodo, or in the README file.

AUTHORS:

- Pieter Belmans (2019-01-27): initial version
- Pieter Belmans (2020-06-16): the version which got assigned a DOI
- Pieter Belmans (2021-08-04): various additions, added unit tests and proper documentation

CHAPTER

ONE

HODGE DIAMONDS

class diamond.**HodgeDiamond**(*parent*, *m*)

This class implements some methods to work with Hodge diamonds.

R = Multivariate Polynomial Ring in x, y over Integer Ring

polynomial ring used internally for the Hodge-Poincaré polynomial and can be used externally to create new polynomials (and thus diamonds)

x = x

variables in the polynomial ring for Hodge-Poincaré polynomials

y = y

variables in the polynomial ring for Hodge-Poincaré polynomials

hide_zeroes = None

static configuration variable for pretty-printing, see *HodgeDiamond.pprint()*

quarter = None

static configuration variable for pretty-printing, see *HodgeDiamond.pprint()*

__init__(*parent*, *m*)

Constructor for a Hodge diamond if you know what you are doing

This just uses the given matrix. It is probably advised to use the class methods

- *HodgeDiamond.from_matrix()*
- *HodgeDiamond.from_polynomial()*

in most cases though.

classmethod from_matrix(*m*, *from_variety=False*)

Construct a Hodge diamond from a matrix

INPUT:

- `m` – square integer matrix representing a Hodge diamond
- `from_variety` (default: False) – whether a check should be performed that it comes from a variety

EXAMPLES:

Hodge diamond of a K3 surface:

```
sage: from diamond import *
sage: S = HodgeDiamond.from_matrix([[1, 0, 1], [0, 20, 0], [1, 0, 1]])
sage: S == K3()
True
```

The following fails as the lack of symmetry prevents a geometric origin:

```
sage: HodgeDiamond.from_matrix([[1, 2], [0, 1]], from_variety=True)
Traceback (most recent call last):
...
AssertionError: The matrix does not
                satisfy the conditions satisfied by the Hodge diamond of a
                smooth projective variety.
```

classmethod from_polynomial(*f*, *from_variety=False*)

Construct a Hodge diamond from a Hodge–Poincaré polynomial

INPUT:

- `f` – an integer polynomial in the ring `HodgeDiamond.R` representing the Hodge–Poincaré polynomial
- `from_variety` (default: False) – whether a check should be performed that it comes from a variety

EXAMPLES:

Hodge diamond of a K3 surface:

```
sage: from diamond import *
sage: x, y = (HodgeDiamond.x, HodgeDiamond.y)
sage: P = 1 + x**2 + 20*x*y + y**2 + x**2 * y**2
sage: S = HodgeDiamond.from_polynomial(P)
sage: S == K3()
True
```

The following fails as the lack of symmetry prevents a geometric origin:

```
sage: HodgeDiamond.from_polynomial(1 + x, from_variety=True)
Traceback (most recent call last):
...
AssertionError: The matrix does not
                satisfy the conditions satisfied by the Hodge diamond of a
                smooth projective variety.
```

property polynomial

The Hodge–Poincaré polynomial describing the Hodge diamond

Getter
returns the Hodge–Poincaré polynomial

Setter
sets the Hodge diamond using a Hodge–Poincaré polynomial

Type
element of *HodgeDiamond.R*

EXAMPLES:

The Hodge–Poincaré polynomial of a K3 surface:

```
sage: from diamond import *
sage: print(K3().polynomial)
x^2*y^2 + x^2 + 20*x*y + y^2 + 1
```

Modifying the Hodge diamond of the projective plane:

```
sage: X = Pn(2)
sage: X.polynomial = X.polynomial + X.x * X.y
sage: print(X)
          1
      0       0
  0       2       0
      0       0
          1
```

property matrix

The matrix describing the Hodge diamond

Getter
returns the matrix

Setter
sets the Hodge diamond using a matrix

Type
square matrix of integers

size()

Internal method to determine the (relevant) size of the Hodge diamond

__eq__(*other*)

Check whether two Hodge diamonds are equal

This compares the Hodge polynomials, not the possibly oversized matrices describing the Hodge diamond.

EXAMPLES:

A quartic surface is a K3 surface:

```
sage: from diamond import *
sage: K3() == hypersurface(4, 2)
True
```

__ne__(*other*)

Check whether two Hodge diamonds are not equal

EXAMPLES:

The projective line is not a genus 2 curve:

```
sage: from diamond import *
sage: Pn(1) != curve(2)
True
```

The point is not the Lefschetz class:

```
sage: point() != lefschetz()
True
```

__hash__()

Hash the Hodge diamond, consistently with equality

Defining `__eq__` without this makes the class unhashable.

EXAMPLES:

```
sage: from diamond import *
sage: len({K3(), hypersurface(4, 2), Pn(2)})
2
```

__pow__(*power*)

Raise a Hodge diamond to a power

This corresponds to iterated multiplication.

INPUT:

- `power` – exponent for the iterated multiplication

EXAMPLES:

The product of 2 K3 surfaces in two ways:

```
sage: from diamond import *
sage: K3()**2 == K3()*K3()
True
```

__call__(*i*, *y=None*)

The calling operator either does a Lefschetz twist, or an evaluation

If one parameter is present, then twist by a power of the Lefschetz Hodge diamond. If two parameters are present, then evaluate the Hodge-Poincaré polynomial

Negative values are allowed to untwist, up to the appropriate power.

INPUT:

- `i` – integer denoting the power of the Lefschetz class, or value for the first variable
- `y` – value of the second variable (default: `None`), if it is non-zero then `i` is reinterpreted as the value of the first variable

EXAMPLES:

The Lefschetz class is by definition the twist of the point:

```
sage: from diamond import *
sage: lefschetz() == point()(1)
True
```

We can reconstruct projective space as a sum of twists of the point:

```
sage: Pn(10) == sum(point()(i) for i in range(11))
True
```

If we supply two parameters we are evaluation the Hodge-Poincaré polynomial, e.g. to find the Euler characteristic:

```
sage: Pn(10)(1, 1) == 11
True
```

__getitem__(*index*)

Get (p, q)th entry of Hodge diamond or the ith row of the Hodge diamond

__str__()

Pretty print Hodge diamond

This gets called when you specifically print the object.

EXAMPLES:

The projective line:

```
sage: from diamond import *
sage: print(Pn(1))
      1
  0       0
      1
```

pprint(*format='table'*, *hide_zeroes=None*, *quarter=None*)

Pretty print the Hodge diamond

INPUT:

- format – output format (default: *"table"*), if table it pretty prints a Hodge diamond; all else defaults to the polynomial
- hide_zeroes (default: False) – whether to hide the zeroes if *"table"* is used as format
- quarter (default: False) – whether to only print the top-left quarter if *"table"* is used for the format

The parameters hide_zeroes and quarter can be set using a static variable, in which case providing them will override this value (if it is set).

EXAMPLES:

The projective line:

```
sage: from diamond import *
sage: Pn(1).pprint()
      1
  0       0
      1
sage: Pn(1).pprint(format="polynomial")
x*y + 1
```

Don't print the zeroes:

```
sage: from diamond import *
sage: (Pn(2) * curve(3)).pprint(hide_zeroes=True)
      1
  3       3
      2
  3       3
      2
  3       3
      1
```

Only print the top-left quarter:

```
sage: from diamond import *
sage: (Pn(2) * curve(3)).pprint(quarter=True)
```

(continues on next page)

(continued from previous page)

```
          1
       3
    0     2
  0     3
```

Only print the top-left quarter whilst hiding zeroes:

```
sage: from diamond import *
sage: (Pn(2) * curve(3)).pprint(hide_zeroes=True, quarter=True)
      1
  3
      2
  3
```

is_hodge_symmetric()

Check whether the Hodge diamond satisfies Hodge symmetry

This checks the equality

$$\mathrm{h}^{p,q}(X) = \mathrm{h}^{q,p}(X)$$

for $p,q=0,\ldots,\dim X$.

Almost all of the constructions provided with the library satisfy Hodge symmetry, because we (somewhat implicitly) work with things which are (or behave like) smooth projective varieties over a field of characteristic zero.

Over the complex numbers this can fail for non-Kähler manifolds, such as the Hopf surface.

In positive characteristic this can fail too, with an example given by classical and singular Enriques surfaces in characteristic 2, see [MR0491720] and Proposition 1.4.2 in [MR0986969]

- [MR0491720] Bombieri–Mumford, Enriques' classification of surfaces in char. p. III.
- [MR0986969] Cossec–Dolgachev, Enriques surfaces I, Progress in Mathematics, 1989

EXAMPLES:

Constructions satisfy this property:

```
sage: from diamond import *
sage: Pn(5).is_hodge_symmetric()
True
```

The Hopf surface over the complex numbers:

```
sage: print(hopf())
          1
      1       0
  0       0       0
      0       1
          1
sage: hopf().is_hodge_symmetric()
False
```

Classical and singular Enriques surfaces in characteristic 2 (which are smooth, despite their name) also have a Hodge diamond violating Hodge symmetry:

```
sage: enriques(two="classical").is_hodge_symmetric()
False
sage: enriques(two="singular").is_hodge_symmetric()
False
sage: enriques(two="supersingular").is_hodge_symmetric()
True
```

is_serre_symmetric()

Check whether the Hodge diamond satisfies Serre symmetry

This checks the equality

$$\mathrm{h}^{p,q}(X) = \mathrm{h}^{\dim X-p,\dim X-q}(X)$$

for \$p,q=0,\ldots,\dim X\$.

Because Serre duality holds for all smooth projective varieties, independent of the characteristic, and also for non-Kähler varieties there are no examples where this condition fails. It can of course fail for motivic pieces, for silly reasons.

EXAMPLES:

The Hilbert scheme of 4 points on a K3 surface satisfies the symmetry:

```
sage: from diamond import *
sage: hilbn(K3(), 4).is_serre_symmetric()
True
```

The Lefschetz diamond fails it for silly reasons:

```
sage: lefschetz().is_serre_symmetric()
False
```

betti()

Betti numbers of the Hodge diamond

This gives an integer vector.

EXAMPLES:

Betti numbers of a K3 surface:

```
sage: from diamond import *
sage: K3().betti()
[1, 0, 22, 0, 1]
```

The second Betti number of the Hilbert scheme of points on a K3 surface is 23, not 22:

```
sage: [hilbn(K3(), n).betti()[2] for n in range(2, 10)]
[23, 23, 23, 23, 23, 23, 23, 23]
```

middle()

Middle cohomology of the Hodge diamond

For smooth projective varieties the middle cohomology sits in degree equal to the dimension.

EXAMPLES:

There is an interesting link between K3 surfaces and cubic fourfolds which can be seen on the level of middle cohomology:

```
sage: from diamond import *
sage: (hypersurface(3, 4) - lefschetz()**2).middle()
[0, 1, 20, 1, 0]
sage: K3().middle()
[1, 20, 1]
```

row(*i*, *truncate=False*)

Get the ith row of the Hodge diamond

For smooth projective varieties these are the Hodge numbers of the Hodge structure on the cohomology in degree *i*.

Alternatively, you can use `HodgeDiamond.__getitem__` with a single index *i* (but then you have to truncate yourself).

INPUT:

- `i` – the row of the Hodge diamond
- `truncate` (default: False) – whether you want to omit leading and trailing zeroes

EXAMPLES:

For a smooth projective variety the middle cohomology is the row sitting in the middle dimension:

```
sage: from diamond import *
sage: hypersurface(3, 4).middle() == hypersurface(3, 4).row(4)
True
```

If you don't want to truncate, `HodgeDiamond.__getitem__` gives the same functionality:

```
sage: from diamond import *
sage: hypersurface(3, 4).row(4) == hypersurface(3, 4)[4]
True
```

For the moduli space of vector bundles on a curve, the cohomology in degree 3 is the same as the cohomology of the curve in degree 1:

```
sage: from diamond import *
sage: moduli_vector_bundles(3, 1, 9).row(3, truncate=True)
[9, 9]
```

Both ends are stripped independently, so a row of zeroes truncates to nothing, and a row which is not symmetric keeps no zeroes:

```
sage: K3().row(1, truncate=True)
[]
sage: hopf().row(1, truncate=True)
[1]
```

signature()

The signature of the Hodge diamond

This is the index of the intersection form on middle cohomology taken with real coefficients. By the Hodge index theorem it is given by the formula in Theorem 6.33 of Voisin's first book on Hodge theory.

$$\sigma = \sum_{p,q=0}^{\dim X} (-1)^p \mathrm{h}^{p,q}$$

This of course only makes sense if the diamond comes from a compact Kähler manifold.

EXAMPLES:

sage: from diamond import * sage: K3().signature() -16

euler()

The topological Euler characteristic of the Hodge diamond

This is the alternating sum of the Betti numbers, so that

$$\chi_{\text{top}} = \sum_{p,q=0}^{\dim X} (-1)^{p+q} \mathrm{h}^{p,q}$$

EXAMPLES:

The Euler characteristic of projective space grows linearly:

```
sage: from diamond import *
sage: [Pn(n).euler() for n in range(10)]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

For Hilbert schemes of points of K3 surfaces these are the coefficients of the series expansion of the Dedekind eta-function, see A006922 in the OEIS:

```
sage: [hilbn(K3(), n).euler() for n in range(10)]
[1, 24, 324, 3200, 25650, 176256, 1073720, 5930496, 30178575, 143184000]
```

holomorphic_euler()

Holomorphic Euler characteristic

This is the Euler characteristic of the structure sheaf, so

$$\chi(X) = \sum_{i=0}^{\dim X} (-1)^i \mathrm{h}^{0,i}(X)$$

EXAMPLES:

For projective space it is 1:

```
sage: from diamond import *
sage: all(Pn(n).holomorphic_euler() == 1 for n in range(10))
True
```

For a hyperkähler variety of dimension $2n$ this number is $n+1$:

```
sage: all(K3n(n).holomorphic_euler() == n+1 for n in range(5))
True
```

It is the alternating sum of *HodgeDiamond.homological_unit()*, so it can differ from the alternating sum of the $\mathrm{h}^{i,0}$ when Hodge symmetry fails:

```
sage: hopf().holomorphic_euler()
0
```

hirzebruch()

Hirzebruch's chi_y genus

For a smooth projective variety X Hirzebruch's χ_y-genus is defined as

$$\chi_y(X) = \sum_{p,q=0}^{\dim X} (-1)^{p+q} \mathrm{h}^{p,q}(X) y^p$$

which shows it is the specialisation of the Hodge-Poincaré polynomial for $x=-1$. A further specialisation to $y=-1$ gives the Euler characteristic.

EXAMPLES:

For a K3 surface we have:

```
sage: from diamond import *
sage: K3().hirzebruch()
2*y^2 - 20*y + 2
sage: K3().hirzebruch().subs(y=-1) == K3().euler()
True
```

For the Hilbert square of a K3 surface we get:

```
sage: hilbn(K3(), 2).hirzebruch()
3*y^4 - 42*y^3 + 234*y^2 - 42*y + 3
sage: hilbn(K3(), 2).hirzebruch().subs(y=-1) == hilbn(K3(), 2).euler()
True
```

homological_unit()

Dimensions of $\mathrm{H}^\bullet(X,\mathcal{O}_X)$

A notion introduced by Abuaf.

hochschild()

Dimensions of the Hochschild homology

Columns of the Hodge diamond are Hochschild homology, by the Hochschild- Kostant-Rosenberg theorem.

hh()

Shorthand for *HodgeDiamond.hochschild()*

arises_from_variety()

Check whether the Hodge diamond can arise from a smooth projective variety

The constraints are:

- satisfy Hodge symmetry
- satisfy Serre symmetry
- there is no Lefschetz twist

is_zero()

Check whether the Hodge diamond is identically zero

lefschetz_power()

Return the twist by the Lefschetz motive that is present

In other words, we see how divisible the Hodge–Poincaré polynomial is with respect to the monomial x^iy^i

dimension()

Dimension of the Hodge diamond

This takes twists by the Lefschetz class into account: we untwist by the maximal power and only then determine how big the diamond is.

EXAMPLES:

A point is 0-dimensional:

```
sage: from diamond import *
sage: point().dimension()
0
```

The Lefschetz diamond is also 0-dimensional:

```
sage: print(lefschetz())
      0
  0       0
      1
sage: lefschetz().dimension()
0
```

Dimension of a non-algebraic surface:

```
sage: inoue().dimension()
2
```

It is symmetric in p and q, also when Hodge symmetry fails:

```
sage: HodgeDiamond.from_matrix([[1, 0, 0], [0, 0, 0], [1, 0, 0]]).dimension()
2
```

The empty diamond has dimension -1:

```
sage: zero().dimension()
-1
```

level()

Compute the level (or complexity) of the Hodge diamond

This is a measure of the width of the non-zero part of the Hodge diamond.

EXAMPLES:

The simplest case is projective space, with level zero:

```
sage: from diamond import *
sage: all(Pn(n).level() == 0 for n in range(10))
True
```

For intersections of 2 quadrics it alternates between zero and one:

```
sage: even = [complete_intersection([2, 2], 2*n) for n in range(10)]
sage: all(Q.level() == 0 for Q in even)
True
sage: odd = [complete_intersection([2, 2], 2*n + 1) for n in range(10)]
```

(continues on next page)

(continued from previous page)

```
sage: all(Q.level() == 1 for Q in odd)
True
```

A Calabi-Yau variety (e.g. a hypersurface of degree $n+1$ in $\mathbb{P}^n$) has maximal level:

```
sage: all(hypersurface(n+2, n).level() == n for n in range(10))
True
```

The empty diamond has level zero:

```
sage: zero().level()
0
```

blowup(*other*, *codim=None*)

Compute Hodge diamond of blowup

No consistency checks are performed, this just naively applies the blowup formula from Hodge theory.

INPUT:

- `other` – Hodge diamond of the center of the blowup
- `codim` – codimension of the center (optional), in case it is not the Hodge diamond of an honest variety

EXAMPLES:

A cubic surface is the blowup of $\mathbb{P}^2$ in 6 points:

```
sage: from diamond import *
sage: Pn(2).blowup(6*point()) == hypersurface(3, 2)
True
```

bundle(*rank*)

Compute the Hodge diamond of a projective bundle

This applies the bundle formula from Hodge theory without any consistency checks.

INPUT:

- `rank`: rank of the vector bundle on `self`

EXAMPLES:

A projective bundle on a point is a projective space:

```
sage: from diamond import *
sage: point().bundle(3) == Pn(2)
True
```

A quadric surface is a $\mathbb{P}^1$-bundle on $\mathbb{P}^1$:

```
sage: Pn(1).bundle(2) == hypersurface(2, 2)
True
```

mirror()

Compute the mirror Hodge diamond

EXAMPLES:

The mirror to a quintic 3-fold is the following:

```
sage: from diamond import *
sage: print(hypersurface(5, 3).mirror())
                1
            0       0
        0       101     0
    1       1       1       1
        0       101     0
            0       0
                1
```

__weakref__

list of weak references to the object

CHAPTER

TWO

HOCHSCHILD HOMOLOGY

class diamond.**HochschildHomology**(*parent*, *L*)

This class implements some methods to work with (the dimensions of) Hochschild homology spaces, associated to the *HodgeDiamond* class.

The documentation is not intended to be complete, as this is mostly for my own sake.

classmethod from_list(*L*)

Constructor for Hochschild homology dimensions from a list.

INPUT:

- **L – a list of integers representing HH_{-n} to** HH_n

EXAMPLES:

```
sage: from diamond import *
sage: HochschildHomology.from_list([1,0,22,0,1])
Hochschild homology vector of dimension 2
```

classmethod from_positive(*L*)

Constructor for Hochschild homology dimensions from a list when only the positive part is given.

INPUT:

- L – a list of integers representing HH_0 to HH_n

EXAMPLES:

```
sage: from diamond import *
sage: HochschildHomology.from_positive([22,0,1])
Hochschild homology vector of dimension 2
```

classmethod from_polynomial(*f*)

Constructor for Hochschild homology dimensions from Hochschild–Poincaré Laurent polynomial

INPUT

- f – the Hochschild–Poincaré Laurent polynomial

EXAMPLES:

```
sage: from diamond import *
sage: x = LaurentPolynomialRing(ZZ, 'x').gen()
sage: HochschildHomology.from_polynomial(x**-2+20+x**2)
Hochschild homology vector of dimension 2
```

A polynomial violating Serre duality is rejected, rather than being reread as a shifted one which happens to be symmetric:

```
sage: HochschildHomology.from_polynomial(x**3)
Traceback (most recent call last):
...
AssertionError: Serre duality is not satisfied
sage: HochschildHomology.from_polynomial(x**2 + 1)
Traceback (most recent call last):
...
AssertionError: Serre duality is not satisfied
```

property polynomial

EXAMPLES:

```
sage: from diamond import *
sage: h = HochschildHomology.from_list([1,0,22,0,1])
sage: h.polynomial
t^-2 + 22 + t^2
```

dimension()

Largest index i such that $\mathrm{HH}_i\neq 0$

EXAMPLES:

```
sage: from diamond import *
sage: h = HochschildHomology.from_list([1,0,22,0,1])
sage: h.dimension()
2
```

is_zero()

EXAMPLES:

```
sage: from diamond import *
sage: h = HochschildHomology.from_list([1,0,22,0,1])
sage: h.is_zero()
False
```

euler()

Euler characteristic of Hochschild homology

EXAMPLES:

```
sage: from diamond import *
sage: h = HochschildHomology.from_list([1,0,22,0,1])
sage: h.euler()
24
```

symmetric_power(*k*)

Hochschild homology of the Ganter–Kapranov symmetric power of a smooth and proper dg category

This is possibly only a heuristic (I didn't check for proofs in the literature) based on the decomposition of Hochschild homology for a quotient stack, as discussed in the paper of Polishchuk–Van den Bergh.

EXAMPLES:

```
sage: from diamond import *
sage: k = K3().hh()
sage: k.symmetric_power(2)
Hochschild homology vector of dimension 4
sage: print(_)
  -4   -3   -2   -1   0     1   2    3   4
  1    0    23   0    276   0   23   0   1
```

sym(*k*)

Shorthand for `HochschildHomology.symmetric_power`

CHAPTER

THREE

CONSTRUCTIONS

3.1 Elementary constructions

diamond.**zero**()

Hodge diamond for the empty space

EXAMPLES:

Zero:

```
sage: from diamond import *
sage: print(zero())
  0
```

diamond.**point**()

Hodge diamond for the point

EXAMPLES:

The point:

```
sage: from diamond import *
sage: print(point())
  1
```

diamond.**lefschetz**()

Hodge diamond for the Lefschetz motive

This is the Hodge-Poincaré polynomial of the affine line.

EXAMPLES:

The affine line:

```
sage: from diamond import *
sage: print(lefschetz())
      0
  0       0
      1
```

We can take powers of it, to get the Hodge-Poincaré polynomial for higher- dimensional affine spaces:

```
sage: print(lefschetz()**3)
              0
```

(continues on next page)

(continued from previous page)

```
          0       0
      0       0       0
  0       0       0       0
      0       0       0
          0       0
              1
```

diamond.**Pn**(*n*)

Hodge diamond for projective space of dimension n

INPUT:

- n: dimension, non-negative integer

EXAMPLES:

The zero-dimensional case is a point:

```
sage: from diamond import *
sage: Pn(0) == point()
True
```

In general projective space is the sum of powers of the Lefschetz class:

```
sage: all(Pn(n) == sum([lefschetz()**i for i in range(n + 1)]) for n in range(1,␣
↪10))
True
```

diamond.**hypersurface**(*degree*, *dimension*)

Shorthand for a complete intersection of the given dimension where $k=1$

diamond.**weighted_hypersurface**(*degree*, *weights*)

Hodge diamond for a weighted hypersurface of degree d in P(w_0,...,w_n)

This implements Theorem 7.2 of Fletcher's notes, Working with weighted complete intersections.

INPUT:

- degree – degree of the hypersurface
- weights – the weights of the weighted projective space, if it is an integer we interpret it as the dimension of P^n

EXAMPLES:

Elliptic curves can be realised as hypersurfaces in 3 ways:

```
sage: from diamond import *
sage: weighted_hypersurface(3, 2) == weighted_hypersurface(3, [1, 1, 1])
True
sage: weighted_hypersurface(3, 2) == curve(1)
True
sage: weighted_hypersurface(4, [1, 1, 2]) == curve(1)
True
sage: weighted_hypersurface(6, [1, 2, 3]) == curve(1)
True
```

The Fano 3-fold 1.1 is a weighted hypersurface:

```
sage: fano_threefold(1, 1) == weighted_hypersurface(6, [1,1,1,1,3])
True
```

If the variety is only quasismooth, not smooth, then we have to interpret the Hodge numbers accordingly. For instance, number 2 on Reid's list of 95 K3s has middle Hodge number 19, because the surface is has one node.

sage: weighted_hypersurface(5, [1, 1, 1, 2]).middle() [1, 19, 1]

diamond.**cyclic_cover**(*ramification_degree*, *cover_degree*, *weights*)

Hodge diamond of a cyclic cover of weighted projective space

Implementation taken from https://github.com/jxxcarlson/math_research/blob/master/hodge.sage.

INPUT:

- `ramification_degree` – degree of the ramification divisor
- `cover_degree` – size of the cover
- `weights` – the weights of the weighted projective space, if it is an integer we interpret it as the dimension of `P^n`

EXAMPLES:

Some K3 surfaces are double covers of $\mathbb{P}^2$ in a sextic curve:

```
sage: from diamond import *
sage: cyclic_cover(6, 2, 2) == K3()
True
```

The Fano 3-fold 1.1 is a double cover of $\mathbb{P}^3$ in a sextic:

```
sage: cyclic_cover(6, 2, 3) == fano_threefold(1, 1)
True
```

diamond.**complete_intersection**(*degrees*, *dimension*)

Hodge diamond for a complete intersection of multidegree $(d_1,\ldots,d_k)$ in $\mathbb{P}^{n+k}$

For a proof, see théorème 2.3 of exposé XI in SGA7.

INPUT:

- `degrees` – the multidegree, if it is an integer we interpret it as a hypersurface
- `dimension` – the dimension of the complete intersection (not of the ambient space)

EXAMPLES:

For multidegrees $(1,\ldots,1) we get a lower-dimension projective space:

```
sage: from diamond import *
sage: complete_intersection(1, 2) == Pn(2)
True
sage: complete_intersection([1, 1], 2) == Pn(2)
True
sage: complete_intersection([1, 1], 5) == Pn(5)
True
```

The Euler characteristics of cubic hypersurfaces:

```
sage: [complete_intersection(3, n).euler() for n in range(10)]
[3, 0, 9, -6, 27, -36, 93, -162, 351, -672]
```

The Euler characteristics of intersections of 2 quadrics:

```
sage: [complete_intersection([2, 2], n).euler() for n in range(10)]
[4, 0, 8, 0, 12, 0, 16, 0, 20, 0]
```

3.2 Curves and moduli spaces of sheaves on them

diamond.**curve**(*genus*)

Hodge diamond for a curve of a given genus.

INPUT:

- genus: the genus of the curve, non-negative integer

EXAMPLE:

A curve of genus 0 is the 1-dimensional projective space:

```
sage: from diamond import *
sage: curve(0) == Pn(1)
True
```

A curve of genus 1 is an abelian variety of dimension 1:

```
sage: curve(1) == abelian(1)
True
```

A curve of genus 2:

```
sage: print(curve(2))
      1
  2       2
      1
```

diamond.**symmetric_power**(*n*, *genus*)

Hodge diamond for the nth symmetric power of a curve of given genus

For the proof, see Example 1.1(1) of [MR2777820]. An earlier reference, probably in Macdonald, should exist.

- [MR2777820] Maxim–Schürmann, Hirzebruch invariants of symmetric products. Topology of algebraic varieties and singularities, 163–177, Contemp. Math., 538, Amer. Math. Soc., 2011.

INPUT:

- n – exponent of the symmetric power
- genus – genus of the curve, a non-negative integer

EXAMPLES:

The symmetric square of a genus 3 curve:

```
sage: from diamond import *
sage: print(symmetric_power(2, 3))
          1
      3       3
  3       10      3
      3       3
          1
```

If $n=1$ we get the curve back:

```
sage: all(symmetric_power(1, g) == curve(g) for g in range(10))
True
```

If $n=0$ we get the point:

```
sage: symmetric_power(0, 4) == point()
True
```

If $n<0$ we have the empty space:

```
sage: symmetric_power(-1, 4) == zero()
True
```

diamond.**jacobian**(*genus*)

Hodge diamond for the Jacobian of a genus g curve

This is an abelian variety of dimension *genus*, so we call *abelian()*

INPUT:

- genus – genus of the curve

EXAMPLES:

The Jacobian of a genus 3 curve:

```
sage: from diamond import *
sage: print(jacobian(3))
              1
          3       3
      3       9       3
  1       9       9       1
      3       9       3
          3       3
              1
```

For the projective line we get a point:

```
sage: jacobian(0) == point()
True
```

The Jacobian of an elliptic curve is isomorphic to it:

```
sage: jacobian(1) == curve(1)
True
```

diamond.**moduli_vector_bundles**(*rank*, *degree*, *genus*)

Hodge diamond for the moduli space of vector bundles of given rank and fixed determinant of given degree on a curve of a given genus.

For the proof, see Corollary 5.1 of [MR1817504].

- [MR1817504] del Baño, On the Chow motive of some moduli spaces. J. Reine Angew. Math. 532 (2001), 105–132.

If the Hodge diamond for the moduli space with non-fixed determinant of degree d is required, this can be obtained by:

```
jacobian(g) * moduli_vector_bundles(r, d, g)
```

INPUT:

- *rank* – rank of the bundles, at least 2
- *degree* – degree of the fixed determinant, coprime to rank
- *genus* – genus of the curve, at least 2

EXAMPLES:

The case of rank 2, degree 1 and genus 2 is famously the intersection of 2 quadrics in $\mathbb{P}^5$:

```
sage: from diamond import *
sage: moduli_vector_bundles(2, 1, 2) == complete_intersection([2, 2], 3)
True
```

diamond.**moduli_higgs_bundles**(*rank*, *degree*, *genus*)

Conjectural Hodge diamond for the moduli space of semistable Higgs bundles of given rank and degree on a curve of a given genus.

This is Conjecture 5.6 of [MR2166085], in the motivic form of Conjecture 2 of [MR2975380], which is what is implemented. Its specialisation to the Poincaré polynomial is a theorem in every rank, by Schiffmann and by Mellit, but the motivic refinement computed here is still a *conjecture*, verified in rank 2 by Hitchin's computation, in rank 3 by Gothen's, and in rank 4 for small genus. You have been warned.

- [MR2166085] Hausel, Mirror symmetry and Langlands duality in the non-abelian Hodge theory of a curve. Geometric methods in algebra and number theory, 193-217, Progr. Math., 235, Birkhauser, 2005.
- [MR2975380] Mozgovoy, Solutions of the motivic ADHM recursion formula. Int. Math. Res. Not. IMRN 2012, no. 18, 4218-4244.

The moduli space is smooth of dimension $2n^2(g-1)+2$, but it is not projective, so what a Hodge diamond can mean for it needs saying. Its cohomology is nevertheless pure, by Theorem 2.1 of [MR2166085]: H^k carries a pure Hodge structure of weight k, so Hodge numbers $\mathrm{h}^{p,q}$ with $p+q=k$ make sense as they do for a projective variety. What is returned are the Hodge numbers of cohomology *with compact support*, which is what the class in the Grothendieck ring computes, so that the entry in position (p,q) is $\dim\mathrm{Gr}^p_F\mathrm{H}^{p+q}_{\mathrm{c}}$. Poincaré duality recovers the ordinary ones as $\mathrm{h}^{p,q}=\mathrm{h}_{\mathrm{c}}^{d-p,d-q}$, with d the dimension. The diamond is Hodge symmetric but not Serre symmetric, and does not arise from a smooth projective variety.

Beware that *HodgeDiamond.dimension()* returns $d/2$ rather than d: the whole class is divisible by $\mathbb{L}^{d/2}$, and the convention there is to untwist by the maximal power of the Lefschetz class before measuring. Use $2n^2(g-1)+2$ for d, or read it off the top corner, which is $\mathrm{h}_{\mathrm{c}}^{d,d}=1$.

The answer does not depend on the degree, only on its being coprime to the rank. Hausel states the conjecture for the moduli space attached to PGL_n, whose formula differs from this one by the factor

`jacobian(genus)(genus)` of the cotangent bundle to the Jacobian. That is a relation between the two formulas, not a decomposition of the variety: the GL_n moduli space is a quotient by the n-torsion of the Jacobian, not a product.

INPUT:

- `rank` – rank of the Higgs bundles, at least 1
- `degree` – degree of the Higgs bundles, coprime to the rank
- `genus` – genus of the curve, at least 1

EXAMPLES:

In rank 1 the moduli space is the cotangent bundle to the Jacobian:

```
sage: from diamond import *
sage: all(moduli_higgs_bundles(1, d, g) == jacobian(g)(g)
....:     for d in range(-2, 3) for g in range(1, 5))
True
```

In genus 1 it is the cotangent bundle to the curve, in every rank coprime to the degree:

```
sage: all(moduli_higgs_bundles(n, 1, 1) == curve(1)(1) for n in range(1, 5))
True
```

Reversing the compactly supported Betti numbers turns compact support into ordinary cohomology, giving the Poincaré polynomial. In rank 2 and genus 2 it is Hitchin's, of degree the dimension 10, so that half of the diamond is empty:

```
sage: P = list(reversed(moduli_higgs_bundles(2, 1, 2).betti()))
sage: P[:11]
[1, 4, 7, 12, 25, 40, 47, 44, 30, 12, 2]
sage: all(b == 0 for b in P[11:])
True
```

The Euler characteristic vanishes, as the moduli space fibres over the cotangent bundle to the Jacobian:

```
sage: moduli_higgs_bundles(2, 1, 3).euler()
0
```

diamond.**seshadris_desingularisation**(*genus*)

Hodge diamond for Seshadri's desingularisation of the moduli space of rank 2 bundles with trivial determinant on a curve of a given genus g.

For the statement, see Corollary 3.18 of [MR1895918].

- [MR1895918] del Baño, On the motive of moduli spaces of rank two vector bundles over a curve. Compositio Math. 131 (2002), 1-30.

INPUT:

- `genus` – the genus g, at least 2

EXAMPLES:

For $g=2$ nothing needs to be desingularised, and the answer is $\mathbb{P}^3$:

```
sage: from diamond import *
sage: seshadris_desingularisation(2) == Pn(3)
True
```

Already for $g=3$ the result is not a familiar variety, so we just check the Euler characteristic:

```
sage: seshadris_desingularisation(3).euler() == 112
True
```

diamond.**narasimhan_ramanans_desingularisation**(*genus*)

Hodge diamond for the Narasimhan–Ramanan desingularisation of the moduli space of rank 2 bundles with trivial determinant on a curve of genus g.

This is the moduli space of Hecke cycles. By Theorem 5.6 of [MR2099191] the morphism from Kirwan's to Seshadri's desingularisation is a composition of two blowdowns, and [MR2122217] identifies the intermediate variety with the moduli space of Hecke cycles. It is the blowup of Seshadri's desingularisation in 2^{2g} copies of $\operatorname{Gr}(3,g)$, which sit in codimension 6.

- [MR2099191] Kiem–Li, Desingularizations of the moduli space of rank 2 bundles over a curve. Math. Ann. 330 (2004), 491-518.
- [MR2122217] Choe–Choy–Kiem, Cohomology of the moduli space of Hecke cycles. Topology 44 (2005), 585-608.

INPUT:

- genus – the genus g, at least 3

EXAMPLES:

It desingularises a moduli space of dimension $3g-3$:

```
sage: from diamond import *
sage: narasimhan_ramanans_desingularisation(4).dimension()
9
```

Seshadri's desingularisation is a blowdown of it, so it has a smaller Euler characteristic:

```
sage: narasimhan_ramanans_desingularisation(3).euler()
432
sage: seshadris_desingularisation(3).euler()
112
```

diamond.**kirwans_desingularisation**(*genus*)

Hodge diamond for Kirwan's desingularisation of the moduli space of rank 2 bundles with trivial determinant on a curve of genus g.

By Theorem 5.6 of [MR2099191] it is the blowup of the Narasimhan–Ramanan desingularisation in 2^{2g} copies of a $\mathbb{P}^{g-2}$-bundle over $\operatorname{Gr}(2,g)$, which sit in codimension 3. Its Betti numbers were computed in [MR2099191] and [MR2122217] by Kirwan's algorithm; the centres of the two blowdowns are of Tate type, so the Hodge numbers follow from those of Seshadri's desingularisation.

- [MR2099191] Kiem–Li, Desingularizations of the moduli space of rank 2 bundles over a curve. Math. Ann. 330 (2004), 491-518.
- [MR2122217] Choe–Choy–Kiem, Cohomology of the moduli space of Hecke cycles. Topology 44 (2005), 585-608.

INPUT:

- genus – the genus g, at least 3

EXAMPLES:

It desingularises a moduli space of dimension $3g-3$:

```
sage: from diamond import *
sage: kirwans_desingularisation(4).dimension()
9
```

Each of the two blowups adds 2^{2g} exceptional divisors to the second Betti number, which is 2 for Seshadri's desingularisation, so it is $2\cdot 2^{2g}+2$ for genus 3:

```
sage: kirwans_desingularisation(3).betti()[2]
130
```

diamond.**moduli_parabolic_vector_bundles_rank_two**(*genus*, *alpha*)

Hodge diamond for the moduli space of parabolic rank 2 bundles with fixed determinant of odd degree on a curve of genus g.

See Corollary 5.34 of [2011.14872].

- [2011.14872] Fu–Hoskins–Pepin Lehalleur, Motives of moduli spaces of bundles on curves via variation of stability and flips

This is not a proof of the formula we implemented per se, but it should be correct. Also, it could be that the choice of weights give something singular / stacky. Then it'll give bad output without warning. You have been warned.

INPUT:

- genus – the genus of the curve
- alpha – the weights of the parabolic bundles

diamond.**quot_scheme_curve**(*genus*, *length*, *rank*)

Hodge diamond for the Quot scheme of zero-dimensional quotients of given length of a vector bundle of given rank on a curve of given genus.

For the proof, see Proposition 4.5 of [1907.00826] (or rather, the reference [Bif89] in there)

- [1907.00826] Bagnarol–Fantechi–Perroni, On the motive of zero-dimensional Quot schemes on a curve

diamond.**brauer_severi**(*g*, *n*, *Ns*)

Hodge diamond of the Brauer–Severi scheme of a hereditary order on a curve

Let $\mathcal{A}$ be a hereditary $\mathcal{O}_C$-order of degree n on a smooth projective curve C of genus g. Its Brauer–Severi scheme (or Artin model) $f\colon\operatorname{BS}(\mathcal{A})\to C$ is a smooth projective variety of dimension n; away from the ramification locus it is a $\mathbb{P}^{n-1}$-bundle over C.

The ramification is encoded by Ns, a list with one entry per ramified point. Each entry is the ramification datum $\mathbf{n}=(n_1,\ldots,n_e)$ at that point, an integer vector summing to n (so e is the ramification index); unramified points are omitted.

The class in the Grothendieck ring is computed by cut-and-paste: a $\mathbb{P}^{n-1}$-bundle over the unramified locus, plus the class of each ramified fibre. The fibre is obtained from Theorem 2.3.29 (intersections of Artin's auxiliary varieties) via inclusion–exclusion, following Chapter 2 of [Baumann].

- [Baumann] Baumann, The geometry of hereditary orders and beyond, PhD thesis, University of Luxembourg, 2025.

INPUT:

- `g` – genus of the base curve
- `n` – degree of the hereditary order
- `Ns` – list of ramification data, one integer vector (summing to `n`) per ramified point

EXAMPLES:

Without ramification we recover a projective bundle over the curve:

```
sage: from diamond import *
sage: brauer_severi(5, 3, []) == curve(5) * Pn(2)
True
```

A conic bundle over $\mathbb{P}^1$ with one nodal fibre is the blowup of $\mathbb{P}^2$ in two points:

```
sage: brauer_severi(0, 2, [(1, 1)]) == Pn(2).blowup(2 * point())
True
```

A degree-2 order over an elliptic curve, ramified in three points:

```
sage: print(brauer_severi(1, 2, [(1, 1)] * 3))
          1
      1       1
  0       5       0
      1       1
          1
```

3.3 Surfaces and moduli spaces of sheaves on them

`diamond.`**`surface`**(*genus*, *irregularity*, *h11*)

Hodge diamond for a surface S with given invariants

These invariants are the geometric genus, the irregularity and the middle Hodge numbers `h11`.

INPUT:

- **`genus` – geometric genus of the surface, $\dim\mathrm{H}^2(S,\mathcal{O}_S)$,**
  a non-negative integer
- **`irregularity` – irregularity of the surface, $\dim\mathrm{H}^1(S,\mathcal{O}_S)$,**
  a non-negative integer
- **`h11` – middle Hodge number, $\dim\mathrm{H}^1(S,\Omega_S^1)$,**
  a non-negative integer

EXAMPLES:

The projective plane:

```
sage: from diamond import *
sage: Pn(2) == surface(0, 0, 1)
True
```

A K3 surface:

```
sage: K3() == surface(1, 0, 20)
True
```

diamond.**ruled**(*genus*)

Hodge diamond for a ruled surface

These are $\mathbb{P}^1$-bundles over a curve of given genus.

INPUT:

- `genus` – genus of the base curve

EXAMPLES:

For genus 0 we get Hirzebruch surfaces, whose Hodge diamond is that of the quadric surface:

```
sage: from diamond import *
sage: ruled(0) == hypersurface(2, 2)
True
```

For higher genus the Hodge diamond looks as follows:

```
sage: print(ruled(5))
          1
      5       5
  0       2       0
      5       5
          1
```

diamond.**K3**()

Hodge diamond for a K3 surface

EXAMPLES:

The K3 surface:

```
sage: from diamond import *
sage: print(K3())
          1
      0       0
  1       20      1
      0       0
          1
```

diamond.**enriques**(*two=None*)

Hodge diamond for an Enriques surface

It is possible to ask for the Hodge diamond of a classical or (super)singular Enriques surface in characteristic 2.

In characteristic 2 the invariants are given in Proposition 1.4.2 of [MR0986969].

- [MR0986969] Cossec–Dolgachev, Enriques surfaces I, Progress in Mathematics, 1989

INPUT:

- **`two` – optional parameter to indicate the type of surface in characteristic 2**
  possible values are *"classical"*, *"singular"*, *"supersingular"*

EXAMPLES:

An ordinary Enriques surface:

```
sage: from diamond import *
sage: print(enriques())
          1
      0       0
  0       10      0
      0       0
          1
```

Enriques surfaces in characteristic 2:

```
sage: print(enriques(two="classical"))
          1
      0       1
  0       12      0
      1       0
          1
sage: print(enriques(two="singular"))
          1
      1       0
  1       10      1
      0       1
          1
sage: print(enriques(two="supersingular"))
          1
      1       1
  1       12      1
      1       1
          1
```

diamond.**hilbn**(*surface*, *n*)

Hodge diamond for Hilbert scheme of n points on a smooth projective surface S

For the proof, see Theorem 2.3.14 of [MR1312161].

- [MR1312161] Göttsche, Hilbert schemes of zero-dimensional subschemes of smooth varieties. Lecture Notes in Mathematics, 1572. Springer-Verlag, Berlin, 1994. x+196 pp.

INPUT:

- S – Hodge diamond for smooth projective surface
- n – number of points

diamond.**nestedhilbn**(*surface*, *n*)

Hodge diamond for the nested Hilbert scheme S^{[n-1,n]}

This is the unique nested Hilbert scheme of a smooth projective surface S which is itself smooth (of dimension $2n$)

diamond.**hilbtwo**(*X*)

Hodge diamond for the Hilbert square of any smooth projective variety

For the proof, see e.g. lemma 2.6 of [MR2506383], or the corollary on page 507 of [MR1382733] (but it can be said to be classical).

- [MR2506383] Muñoz–Ortega–Vázquez-Gallo, Hodge polynomials of the moduli spaces of triples of rank (2,2). Q. J. Math. 60 (2009), no. 2, 235–272.

- [MR1382733] Cheah, On the cohomology of Hilbert schemes of points, J. Algebraic Geom. 5 (1996), no. 3, 479-511

INPUT:

- `X` - Hodge diamond of the smooth projective variety

EXAMPLES:

We recover the Hilbert square of a surface:

```
sage: from diamond import *
sage: hilbtwo(K3()) == hilbn(K3(), 2)
True
```

diamond.**hilbthree**(*X*)

Hodge diamond of the Hilbert cube of any smooth projective variety

The corollary on page 507 of [MR1382733].

- [MR1382733] Cheah, On the cohomology of Hilbert schemes of points, J. Algebraic Geom. 5 (1996), no. 3, 479-511

INPUT:

- `X` - Hodge diamond of the smooth projective variety

EXAMPLES:

We recover the Hilbert cube of a surface:

```
sage: from diamond import *
sage: hilbthree(K3()) == hilbn(K3(), 3)
True
```

diamond.**enriques_hilbn_cover**(*n*)

Hodge diamond of the universal covering space of the Hilbert scheme of `n` points on a complex Enriques surface S

By Theorem 3.1 of [MR2863907] the Hilbert scheme has fundamental group of order 2, so the covering is étale of degree 2, and by Proposition 1.6 of [MR2578804] it is a Calabi-Yau variety of dimension $2n$ for $ngeq 2$.

Let L be the non-trivial rank one local system on S, the one belonging to its K3 double cover, so that $mathrm{H}^bullet(S,L)$ is the anti-invariant part of the cohomology of that K3 surface. By Corollary 1.3 of [MR2578804] the cohomology of the covering space is $mathrm{H}^bullet(S^{[n]})oplusmathrm{H}^bullet(S^{[n]},L^{[n]})$, and the twisted summand is Göttsche's product with the Hodge numbers of S replaced by those of $mathrm{H}^bullet(S,L^{otimes k})$ in the factor of multiplicity k. Since L has order 2 this changes only the factors of odd multiplicity.

- [MR2578804] Nieper-Wißkirchen, Twisted cohomology of the Hilbert schemes of points on surfaces. Doc. Math. 14 (2009), 749–770.
- [MR2863907] Oguiso–Schröer, Enriques manifolds. J. Reine Angew. Math. 661 (2011), 215–235.

For $n=2$ the Hodge numbers are also computed in [MR3778120]. They agree with the ones below except for $mathrm{h}^{2,2}$, which is printed as 131 there. The Euler characteristic favours the 132 below: an étale double cover doubles the Euler characteristic, so it has to be twice the 90 of `hilbn(enriques(), 2)`, whereas 131 gives 179.

- [MR3778120] Hayashi, Universal covering Calabi-Yau manifolds of the Hilbert schemes of n-points of Enriques surfaces. Asian J. Math. 21 (2017), no. 6, 1099–1120.

INPUT:

- n – number of points

EXAMPLES:

For 1 point the covering space is the K3 surface covering the Enriques surface:

```
sage: from diamond import *
sage: enriques_hilbn_cover(1) == K3()
True
```

The Calabi-Yau fourfold covering the Hilbert square:

```
sage: print(enriques_hilbn_cover(2))
                    1
                0       0
            0       12      0
        0       0       0       0
  1       10      132     10      1
        0       0       0       0
            0       12      0
                0       0
                    1
```

The covering map is étale of degree 2, so it doubles the Euler characteristic:

```
sage: S = enriques()
sage: all(enriques_hilbn_cover(n).euler() == 2 * hilbn(S, n).euler() for n in␣
↪range(1, 6))
True
```

The second Betti number is 12 for 2 points, but the covering involution acts trivially on the second cohomology from 3 points on, so it drops to 11 there, as in [MR3778120]:

```
sage: [enriques_hilbn_cover(n).betti()[2] for n in range(1, 6)]
[22, 12, 11, 11, 11]
```

diamond.**hopf**()

Hodge diamond of a Hopf surface

diamond.**inoue**()

Hodge diamond of an Inoue surface

diamond.**kodaira_primary**()

Hodge diamond of a primary Kodaira surface

These are non-Kähler surfaces with $\mathrm{b}_1=3$, so Hodge symmetry fails: $\mathrm{h}^{1,0}=1$ and $\mathrm{h}^{0,1}=2$.

diamond.**kodaira_secondary**()

Hodge diamond of a secondary Kodaira surface

These are non-Kähler surfaces with $\mathrm{b}_1=1$ and $\mathrm{b}_2=0$, so they have the same Hodge diamond as the Hopf and Inoue surfaces.

3.4 Abelian varieties and related objects

diamond.**abelian**(*dimension*)

Hodge diamond for an abelian variety of a given dimension.

The gth power of an elliptic curve is an abelian variety of the given dimension, so we just use this computation method.

INPUT:

- `dimension` – dimension of the abelian variety

EXAMPLES:

A 1-dimensional abelian variety is an elliptic curve:

```
sage: from diamond import *
sage: abelian(1) == curve(1)
True
```

A 2-dimensional abelian variety is a surface with known Hodge numbers:

```
sage: abelian(2) == surface(1, 2, 4)
True
```

diamond.**kummer_resolution**(*dimension*)

Hodge diamond for the standard resolution of the Kummer variety of an abelian variety of a given dimension.

There's an invariant part (Hodge numbers of even degree) and the resolution of the 2^2g singularities is added.

INPUT:

- `dimension` – dimension of the abelian variety taken as input

EXAMPLES:

The Kummer resolution of the involution on an abelian surface is a K3 surface:

```
sage: from diamond import *
sage: kummer_resolution(2) == K3()
True
```

3.5 Fano varieties

These are Hodge diamonds of Fano varieties in the sense that their anticanonical bundle is ample.

The term "Fano variety" can also mean a variety parametrising linear subspaces on another variety. Some of these are Fano in the first sense, others are not (always). See e.g. *diamond.fano_variety_lines_cubic()*.

diamond.**fano_threefold**(*rho*, *ID*)

Hodge diamond of a Fano threefold

INPUT:

- `rho` - Picard rank
- `ID` - numbering from the Mori-Mukai classification

EXAMPLES:

The 17th Fano 3-fold of rank 1 is projective threespace:

```
sage: from diamond import *
sage: print(fano_threefold(1, 17))
              1
          0       0
      0       1       0
  0       0       0       0
      0       1       0
          0       0
              1
```

The 4th Fano 3-fold of rank 1 is an intersection of 3 quadrics:

```
sage: fano_threefold(1, 4) == complete_intersection((2, 2, 2), 3)
True
```

The 27th Fano 3-fold of rank 3 is the triple product of projective lines:

```
sage: fano_threefold(3, 27) == Pn(1)**3
True
```

diamond.**gushel_mukai**(*n*)

Hodge diamond for a smooth n-dimensional Gushel–Mukai variety

See Proposition 3.1 of [1605.05648v3].

- [1605.05648v3] Debarre–Kuznetsov, Gushel-Mukai varieties: linear spaces and periods

INPUT:

- n - the dimension, where $n=1,\ldots,6$

diamond.**fano_variety_intersection_quadrics_even**(*g*, *k*)

Hodge diamond for the Fano variety of $i-1$-planes on the intersection of two quadrics in $\mathbb{P}^{2g+1}$, using [1510.05986v3].

- [1510.05986v3] Chen–Vilonen–Xue, Springer correspondence, hyperelliptic curves, and cohomology of Fano varieties

INPUT:

- g – half of the dimension of the quadrics
- k – dimension of the linear subspaces on the intersection of quadrics, at most $g-1$

EXAMPLES:

For $k=0$ we have the intersection of two quadrics:

```
sage: from diamond import *
sage: fano_variety_intersection_quadrics_even(2, 0) == complete_intersection([2, 2],
↪ 2)
True
sage: fano_variety_intersection_quadrics_even(5, 0) == complete_intersection([2, 2],
↪ 8)
True
```

We have that for $k = g-2$ we get the moduli space of parabolic bundles on $\mathbb{P}^1$ with weight $1/2$ in $2g+3$ points:

```
sage: from diamond import *
sage: moduli_parabolic_vector_bundles_rank_two(0, [1/2]*5) == fano_variety_
↪intersection_quadrics_even(2, 0)
True
sage: moduli_parabolic_vector_bundles_rank_two(0, [1/2]*9) == fano_variety_
↪intersection_quadrics_even(4, 2)
True
```

For $k=g-1$ we get a finite reduced scheme of length 4^g:

```
sage: from diamond import *
sage: print(fano_variety_intersection_quadrics_even(4, 3))
  256
```

diamond.**fano_variety_intersection_quadrics_odd**(*g*, *k*)

Hodge diamond for the Fano variety of k-planes on the intersection of two quadrics in $\mathbb{P}^{2g+1}$, using [MR3689749].

We have that for $k=g-2$ we get M_C(2,L) as above, for deg L odd.

- [MR3689749] Chen–Vilonen–Xue, On the cohomology of Fano varieties and the Springer correspondence, Adv. Math. 318 (2017), 515–533.

EXAMPLES:

For $k=0$ we have the intersection of two quadrics:

```
sage: from diamond import *
sage: fano_variety_intersection_quadrics_odd(2, 0) == complete_intersection([2, 2],␣
↪3)
True
sage: fano_variety_intersection_quadrics_odd(5, 0) == complete_intersection([2, 2],␣
↪9)
True
```

For $k=g-2$ we recover the moduli space of rank 2 bundles with odd determinant on a curve of genus g:

```
sage: from diamond import *
sage: fano_variety_intersection_quadrics_odd(11, 9) == moduli_vector_bundles(2, 1,␣
↪11)
True
```

For $k=g-1$ it is the Jacobian of C:

```
sage: from diamond import *
sage: fano_variety_intersection_quadrics_odd(12, 11) == jacobian(12)
True
```

For other k it is an interesting variety:

```
sage: from diamond import *
sage: fano_variety_intersection_quadrics_odd(12, 9)
Hodge diamond of size 51 and dimension 50
```

diamond.**fano_variety_lines_cubic**(*n*)

Hodge diamond for the Fano variety of lines on a smooth n-dimensional cubic hypersurface.

This follows from the "beautiful formula" or X-F(X)-relation due to Galkin–Shinder, Theorem 5.1 of [1405.5154v2].

- [1405.5154v2] Galkin–Shinder, The Fano variety of lines and rationality problem for a cubic hypersurface

INPUT:

- `n` – the dimension, where `n` is at least 2

EXAMPLES:

There are 27 lines on a cubic surface:

```
sage: from diamond import *
sage: fano_variety_lines_cubic(2) == 27*point()
True
```

The Fano surface of lines on a cubic threefold is a surface of general type:

```
sage: fano_variety_lines_cubic(3) == surface(10, 5, 25)
True
```

The Fano fourfold of lines on a cubic fourfold is deformation equivalent to the Hilbert square on a K3 surface:

```
sage: fano_variety_lines_cubic(4) == hilbn(K3(), 2)
True
```

3.6 Homogeneous varieties and closely related constructions

These are also all Fano varieties, but they are grouped together because of their similar origin.

`diamond.`**`partial_flag_variety`**(*D*, *I*)

Hodge diamond of a partial flag variety G/P

This is computed by counting the number of Schubert cells in the appropriate dimension.

INPUT:

- `D` – Dynkin type
- `I` – indices of vertices to be omitted in defining the parabolic subgroup

EXAMPLES:

An absolute baby case is projective space:

```
sage: from diamond import *
sage: partial_flag_variety("A5", [2,3,4,5]) == Pn(5)
True
```

The next easiest case are quadrics:

```
sage: partial_flag_variety("B5", [2,3,4,5]) == hypersurface(2, 9)
True
sage: partial_flag_variety("D5", [2,3,4,5]) == hypersurface(2, 8)
True
```

diamond.**generalised_grassmannian**(*D*, *k*)

Hodge diamond of the generalised Grassmannian of type D modulo the maximal parabolic subgroup P_k.

This is just shorthand for `partial_flag_variety(D, I)()` where `I` is the complement of a singleton.

INPUT:

- `D` – Dynkin type
- `k` – the vertex in the Dynkin diagram defining the maximal parabolic

diamond.**grassmannian**(*k*, *n*)

Hodge diamond of the Grassmannian $\operatorname{Gr}(k,n)$ of k-dimensional subspaces in an n-dimensional vector space

INPUT:

- `k` – dimension of the subspaces
- `n` – dimension of the ambient vector space

EXAMPLES:

Grassmannians are projective spaces if *k* is one or *n-1*:

```
sage: from diamond import *
sage: grassmannian(1, 5) == Pn(4)
True
sage: grassmannian(7, 8) == Pn(7)
True
```

The Grassmannian of 2-planes in a 4-dimensional vector space is the Kleiin quadric:

```
sage: grassmannian(2, 4) == hypersurface(2, 4)
True
```

diamond.**orthogonal_grassmannian**(*k*, *n*)

Hodge diamond of the orthogonal Grassmannian $\operatorname{OGr}(k, n)$ of k-dimensional subspaces in an n-dimensional vector space isotropic with respect to a non-degenerate symmetric bilinear form

INPUT:

- `k` – dimension of the subspaces
- `n` – dimension of the ambient vector space

EXAMPLES:

Isotropic lines form a quadric hypersurface:

```
sage: from diamond import *
sage: all(orthogonal_grassmannian(1, n) == hypersurface(2, n - 2)
....:     for n in range(4, 10))
True
```

The dimension is $k(n-k)-\binom{k+1}{2}$:

```
sage: [orthogonal_grassmannian(k, 10).dimension() for k in range(1, 5)]
[8, 13, 15, 14]
```

diamond.**symplectic_grassmannian**(*k*, *n*)

Hodge diamond of the symplectic Grassmannian $\operatorname{SGr}(k, n)$ of k-dimensional subspaces in an n-dimensional vector space isotropic with respect to a non-degenerate skew-symmetric bilinear form

INPUT:

- k – dimension of the subspaces
- n – dimension of the ambient vector space

EXAMPLES:

Every line is isotropic for a symplectic form, so for $k=1$ we get projective space:

```
sage: from diamond import *
sage: all(symplectic_grassmannian(1, 2*n) == Pn(2*n - 1)
....:     for n in range(1, 6))
True
```

The dimension is $k(n-k)-\binom{k}{2}$:

```
sage: [symplectic_grassmannian(k, 8).dimension() for k in range(1, 5)]
[7, 11, 12, 10]
```

diamond.**lagrangian_grassmannian**(*n*)

Shorthand for the symplectic Grassmannian of Lagrangian subspaces

diamond.**horospherical**(*D*, *y=0*, *z=0*)

Horospherical varieties as discussed in [1803.05063], with labelling and notation as in op. cit.

INPUT:

- **D: either a Dynkin type from the (small) list of allowed types**
  in the classification _or_ a plaintext label from X1(n), X2, X3(n,m), X4, X5
- y: index for the parabolic subgroup for Y, see classification
- z: index for the parabolic subgroup for the closed orbit Z

y and z must be omitted if a plaintext description is given.

- [1803.05063] Gonzales–Pech–Perrin–Samokhin, Geometry of horospherical varieties of Picard rank one

diamond.**odd_symplectic_grassmannian**(*k*, *n*)

Hodge diamond of the odd symplectic Grassmannian $\operatorname{SGr}(k,n)$

Here n is odd. This is just shorthand for a call to *horospherical()* for type C, with parameters $\lfloor n/2\rfloor$, and Y and Z determined by k and $k - 1$.

EXAMPLES:

For $k=1$ every line is isotropic, so we recover projective space:

```
sage: from diamond import *
sage: odd_symplectic_grassmannian(1, 5) == Pn(4)
True
```

3.7 Moduli spaces attached to quivers

diamond.**quiver_moduli**(*Q*, *d*, ***kwargs*)

Hodge diamond for the moduli space of semistable quiver representations for a quiver Q, dimension vector d, and slope-stability condition mu.

Taken from Corollary 6.9 of [MR1974891]

- [MR1974891] Reineke, The Harder-Narasimhan system in quantum groups and cohomology of quiver moduli.

INPUT:

- `Q` – adjacency matrix of an acyclic quiver
- `d` – dimension vector
- `mu` – stability condition, these can be produced using *slope()*, if left unspecified then the canonical stability condition is used

The canonical stability condition for dimension vector *d* is given by the antisymmetrised Euler form pairing with *d*.

EXAMPLES:

Let's consider moduli spaces for the Kronecker quiver:

```
sage: from diamond import *
sage: def kronecker(d): return matrix([[0, d], [0, 0]])
```

For the 2-Kronecker quiver and dimension vector *(1,1)* a representation is given by 2 scalars, and the stability condition *(1,-1)* encodes that they are not both zero. This way we obtain the projective line:

```
sage: quiver_moduli(kronecker(2), (1, 1), mu=slope((1, -1))) == Pn(1)
True
```

There is only one relevant stability chamber here, which contains the canonical stability condition. Thus we get the same result not specifying the stability function:

```
sage: quiver_moduli(kronecker(2), (1, 1)) == Pn(1)
True
```

Similar to the first example, the d-Kronecker quiver gives rise to projective spaces:

```
sage: all(quiver_moduli(kronecker(d), (1, 1)) == Pn(d - 1) for d in range(3, 10))
True
```

We can also realise Grassmannians using the d-Kronecker quiver, for dimension vector $(1,k)$ and stability condition $(k,1)$ (or the canonical stability condition) we get the Grassmannian $\operatorname{Gr}(k,d)$:

```
sage: quiver_moduli(kronecker(4), (1, 2), mu=slope((2, 1))) == grassmannian(2, 4)
True
sage: quiver_moduli(kronecker(7), (1, 3)) == grassmannian(3, 7)
True
```

Any stability function in the same chamber gives the same variety:

```
sage: quiver_moduli(kronecker(7), (1, 3)) == quiver_moduli(kronecker(7), (1, 3),␣
↪mu=slope((1, -1)))
True
```

The following is an example of wall-crossing, and thus different varieties:

```
sage: M = matrix([[0, 1, 1], [0, 0, 2], [0, 0, 0]])
sage: quiver_moduli(M, (1, 1, 1)) == Pn(2).blowup(point())
True
sage: quiver_moduli(M, (1, 1, 1), mu=slope((1, 0, 0))) == Pn(2)
True
```

The flag variety $\operatorname{Fl}(n,r_1,\ldots,r_s)$ is also a quiver moduli space, for A_s quiver prefixed with an n-Kronecker quiver, dimension vector $(1,r_1,\ldots,r_s)$ with $r_1>\ldots>r_s$ and stability condition the indicator function at the first vertex:

```
sage: def flags(n, s): return matrix(ZZ, s+1, s+1, lambda i,j: 0 if i != j-1 else␣
↪(n if i == 0 else 1))
sage: quiver_moduli(flags(4, 1), (1, 1)) == Pn(3)
True
sage: quiver_moduli(flags(3, 2), (1, 2, 1)) == fano_threefold(2, 32)
True
sage: quiver_moduli(flags(5, 3), (1, 4, 3, 1)) == partial_flag_variety("A4", [2])
True
```

diamond.**slope**(*Theta*)

Helper function to turn stability for quiver representations in slope stability

3.8 Hyperkähler varieties

diamond.**K3n**(*n*)

Hodge diamond of the Hilbert scheme of n points on a K3 surface

This is the first family of hyperkähler varieties, constructed by Beauville.

INPUT:

- `n` – number of points

EXAMPLES:

For $n=1$ we have a K3 surface:

```
sage: from diamond import *
sage: K3n(1) == K3()
True
```

For $n\geq 2$ we have second Betti number 23:

```
sage: all(K3n(n).betti()[2] == 23 for n in range(2, 5))
True
```

diamond.**generalised_kummer**(*n*)

Hodge diamond of the nth generalised Kummer variety

For the proof, see Corollary 1 of [MR1219901].

- [MR1219901] Göttsche–Soergel, Perverse sheaves and the cohomology of Hilbert schemes of smooth algebraic surfaces. Math. Ann. 296 (1993), no. 2, 235–245.

EXAMPLES:

The first generalised Kummer is just a point:

```
sage: from diamond import *
sage: generalised_kummer(1) == point()
True
```

The second generalised Kummer is the Kummer K3 surface:

```
sage: generalised_kummer(2) == K3()
True
```

The higher generalised Kummers are hyperkähler varieties with second Betti number 7:

```
sage: all(generalised_kummer(n).betti()[2] == 7 for n in range(3, 10))
True
```

diamond.**ogrady6**()

Hodge diamond for O'Grady's exceptional 6-dimensional hyperkähler variety

For the proof, see Theorem 1.1 of [MR3798592].

- [MR3798592] Mongardi–Rapagnetta–Saccà, The Hodge diamond of O'Grady's six-dimensional example. Compos. Math. 154 (2018), no. 5, 984–1013.

EXAMPLES:

The second Betti number is 8:

```
sage: from diamond import *
sage: ogrady6().betti()[2] == 8
True
```

diamond.**ogrady10**()

Hodge diamond for O'Grady's exceptional 10-dimensional hyperkähler variety

For the proof, see theorem A of [1905.03217]

- [1905.03217] de Cataldo–Rapagnetta–Saccà, The Hodge numbers of O'Grady 10 via Ngô strings

EXAMPLES:

The second Betti number is 24:

```
sage: from diamond import *
sage: ogrady10().betti()[2] == 24
True
```

3.9 Other

diamond.**Mzeronbar**(*n*)

Hodge diamond for the moduli space of n-pointed stable curves of genus 0

Taken from (0.12) in [MR1363064]. Keel's original paper has a recursion on page 550, but that seems to not work.

- [MR1363064] Manin, Generating functions in algebraic geometry and sums over trees

EXAMPLES:

The first few cases are a point, the projective line, and the blowup of $\mathbb{P}^2$ in 4 points:

```
sage: from diamond import *
sage: Mzeronbar(3) == point()
True
sage: Mzeronbar(4) == Pn(1)
True
sage: Mzeronbar(5) == Pn(2).blowup(4*point())
True
```

PYTHON MODULE INDEX

d

diamond.diamond, 1

INDEX

Symbols

__call__() (*diamond.HodgeDiamond method*), 6
__eq__() (*diamond.HodgeDiamond method*), 5
__getitem__() (*diamond.HodgeDiamond method*), 6
__hash__() (*diamond.HodgeDiamond method*), 5
__init__() (*diamond.HodgeDiamond method*), 3
__ne__() (*diamond.HodgeDiamond method*), 5
__pow__() (*diamond.HodgeDiamond method*), 6
__str__() (*diamond.HodgeDiamond method*), 6
__weakref__ (*diamond.HodgeDiamond attribute*), 15

A

abelian() (*in module diamond*), 35
arises_from_variety() (*diamond.HodgeDiamond method*), 12

B

betti() (*diamond.HodgeDiamond method*), 9
blowup() (*diamond.HodgeDiamond method*), 14
brauer_severi() (*in module diamond*), 29
bundle() (*diamond.HodgeDiamond method*), 14

C

complete_intersection() (*in module diamond*), 23
curve() (*in module diamond*), 24
cyclic_cover() (*in module diamond*), 23

D

diamond.diamond
    module, 1
dimension() (*diamond.HochschildHomology method*), 18
dimension() (*diamond.HodgeDiamond method*), 12

E

enriques() (*in module diamond*), 31
enriques_hilbn_cover() (*in module diamond*), 33
euler() (*diamond.HochschildHomology method*), 18
euler() (*diamond.HodgeDiamond method*), 11

F

fano_threefold() (*in module diamond*), 35
fano_variety_intersection_quadrics_even() (*in module diamond*), 36
fano_variety_intersection_quadrics_odd() (*in module diamond*), 37
fano_variety_lines_cubic() (*in module diamond*), 37
from_list() (*diamond.HochschildHomology class method*), 17
from_matrix() (*diamond.HodgeDiamond class method*), 3
from_polynomial() (*diamond.HochschildHomology class method*), 17
from_polynomial() (*diamond.HodgeDiamond class method*), 4
from_positive() (*diamond.HochschildHomology class method*), 17

G

generalised_grassmannian() (*in module diamond*), 38
generalised_kummer() (*in module diamond*), 42
grassmannian() (*in module diamond*), 39
gushel_mukai() (*in module diamond*), 36

H

hh() (*diamond.HodgeDiamond method*), 12
hide_zeroes (*diamond.HodgeDiamond attribute*), 3
hilbn() (*in module diamond*), 32
hilbthree() (*in module diamond*), 33
hilbtwo() (*in module diamond*), 32
hirzebruch() (*diamond.HodgeDiamond method*), 11
hochschild() (*diamond.HodgeDiamond method*), 12
HochschildHomology (*class in diamond*), 17
HodgeDiamond (*class in diamond*), 3
holomorphic_euler() (*diamond.HodgeDiamond method*), 11
homological_unit() (*diamond.HodgeDiamond method*), 12
hopf() (*in module diamond*), 34
horospherical() (*in module diamond*), 40
hypersurface() (*in module diamond*), 22

I

inoue() (*in module diamond*), 34
is_hodge_symmetric() (*diamond.HodgeDiamond method*), 8
is_serre_symmetric() (*diamond.HodgeDiamond method*), 9
is_zero() (*diamond.HochschildHomology method*), 18
is_zero() (*diamond.HodgeDiamond method*), 12

J

jacobian() (*in module diamond*), 25

K

K3() (*in module diamond*), 31
K3n() (*in module diamond*), 42
kirwans_desingularisation() (*in module diamond*), 28
kodaira_primary() (*in module diamond*), 34
kodaira_secondary() (*in module diamond*), 34
kummer_resolution() (*in module diamond*), 35

L

lagrangian_grassmannian() (*in module diamond*), 40
lefschetz() (*in module diamond*), 21
lefschetz_power() (*diamond.HodgeDiamond method*), 12
level() (*diamond.HodgeDiamond method*), 13

M

matrix (*diamond.HodgeDiamond property*), 5
middle() (*diamond.HodgeDiamond method*), 9
mirror() (*diamond.HodgeDiamond method*), 14
module
    diamond.diamond, 1
moduli_higgs_bundles() (*in module diamond*), 26
moduli_parabolic_vector_bundles_rank_two() (*in module diamond*), 29
moduli_vector_bundles() (*in module diamond*), 25
Mzeronbar() (*in module diamond*), 43

N

narasimhan_ramanans_desingularisation() (*in module diamond*), 28
nestedhilbn() (*in module diamond*), 32

O

odd_symplectic_grassmannian() (*in module diamond*), 40
ogrady10() (*in module diamond*), 43
ogrady6() (*in module diamond*), 43
orthogonal_grassmannian() (*in module diamond*), 39

P

partial_flag_variety() (*in module diamond*), 38
Pn() (*in module diamond*), 22
point() (*in module diamond*), 21
polynomial (*diamond.HochschildHomology property*), 18
polynomial (*diamond.HodgeDiamond property*), 4
pprint() (*diamond.HodgeDiamond method*), 7

Q

quarter (*diamond.HodgeDiamond attribute*), 3
quiver_moduli() (*in module diamond*), 41
quot_scheme_curve() (*in module diamond*), 29

R

R (*diamond.HodgeDiamond attribute*), 3
row() (*diamond.HodgeDiamond method*), 10
ruled() (*in module diamond*), 30

S

seshadris_desingularisation() (*in module diamond*), 27
signature() (*diamond.HodgeDiamond method*), 10
size() (*diamond.HodgeDiamond method*), 5
slope() (*in module diamond*), 42
surface() (*in module diamond*), 30
sym() (*diamond.HochschildHomology method*), 19
symmetric_power() (*diamond.HochschildHomology method*), 18
symmetric_power() (*in module diamond*), 24
symplectic_grassmannian() (*in module diamond*), 39

W

weighted_hypersurface() (*in module diamond*), 22

X

x (*diamond.HodgeDiamond attribute*), 3

Y

y (*diamond.HodgeDiamond attribute*), 3

Z

zero() (*in module diamond*), 21